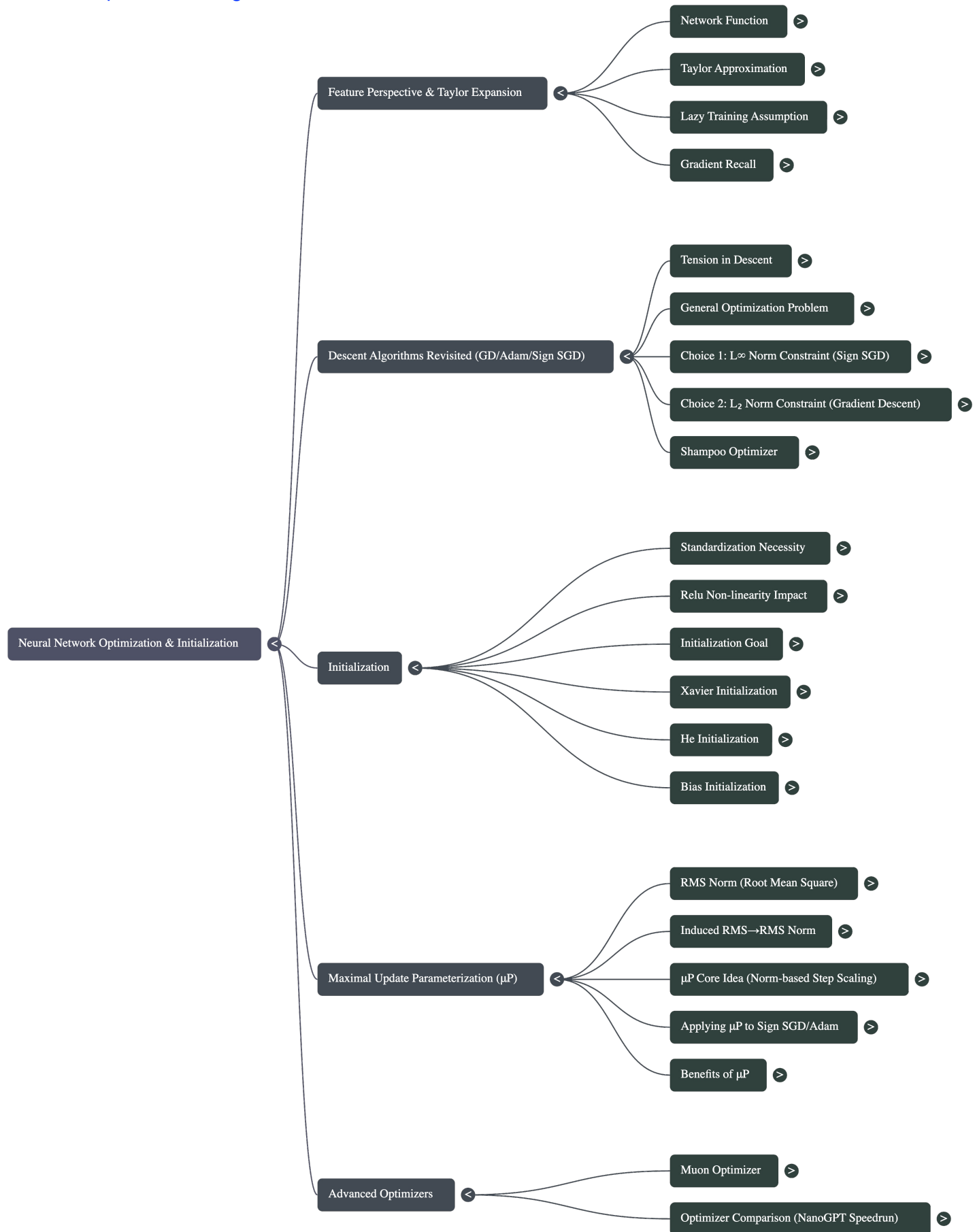


NotebookLM generated a mind map that easily allowed me to figure out WHAT questions to ask. This was the first thing I generated before starting to ask questions, because it gave me a solid understand of what topics were covered in the sources I inputted, which were the following lectures (and corresponding lecture notes):

- Optimization: Adam, taking a locally linear perspective, and what is a feature anyway
- Optimizers
- Optimizers: insights from induced matrix norms



NotebookLM generated a coherent and structured outline, but it blends together ideas from several separate sources (initialization, optimizers, NTK, Muon, maximal update parameterization).

Some explanations are accurate; others are imprecise or flatten subtle distinctions.

The outline is usable as a roadmap, but not reliable as a final learning resource without correction.

You: Can you give me an overview of ALL the concepts listed in the sources I've provided? Perhaps generate a roadmap of topics covered that I can use to ask further enhanced questions.

NotebookLM:

The concepts covered in the provided sources focus primarily on Initialization strategies for neural networks and the theoretical foundations and development of Optimization algorithms, particularly through the lens of local linear approximation and scaling properties.

Here is a roadmap of the key topics and concepts discussed in the sources:

-Solid description of Taylor expansion of the network around parameters.
-The connection between lazy training and NTK is conceptually correct.

I. Feature Perspective and Local Linear Approximation

This section explores how neural network training can be understood through linearization and the features derived from the network's gradient.

- Feature Perspective and Taylor Expansion: The network output $f(x, \Theta)$ is analyzed as a function of the parameters Θ . The Taylor expansion is used to approximate the network's output when parameters change by a small amount $\Delta \Theta$ (a "linear approximation at the network").
- Lazy Training Assumption: This assumes that parameter weights move by very small amounts, allowing the Taylor approximation to hold true in a local neighborhood.
- Gradient and Features: The linear term in the approximation, $\langle \nabla f, \Delta \Theta \rangle$, shows that the entries of the gradient $\nabla \Theta$ function as "features".
- Kernel/Feature Learning Perspective (NTK): Under lazy training assumptions (often observed when using Adam), models barely change from random initialization, linking the process to the Kernel/feature learning perspective and the Neural Tangent Kernel (NTK).

II. Initialization Techniques

Correct high-level goals of Xavier and He initialization.
Good explanation that ReLU kills half the signal → He scaling maintains variance.

Proper initialization is crucial for ensuring that network activations maintain desirable properties (like being standardized) as signals propagate through the layers.

- Initialization Overview: The fundamental challenge is that initializing weights to zero results in zero gradients, preventing weights from ever moving.
- Standardization: Data standardization is performed so that features are on similar orders of magnitude, improving conditioning and preventing numerical issues.
- Xavier Initialization (Weights): The goal is to ensure the input to the non-linearity starts "standardized" (approximately zero-mean, unit-variance). This initialization suggests scaling weights $W_i \sim N(0, 1/d)$, where d is the fan-in.
- He Initialization (Weights): This is specifically used for ReLU non-linearities. Since ReLU outputs are zero approximately half the time, the weights are scaled as $W_i \sim N(0, 2/d)$ to maintain signal variance.
- Bias Initialization: Options include initializing biases at 0, using a small number like 0.01, or treating them under the Xavier initialization framework.
- Impact of Poor Initialization: If data ranges prevent the non-linearity's "corner" (like the ReLU corner) from landing in the data range, the network loses expressive power.
- ReLU and Cauchy Distribution: When initializing weights (w) and biases (b) from $N(0, 1)$, the critical ratio $-b/w$ follows a Cauchy distribution, which is "0-mean" but has infinite variance.

III. Optimization Algorithms (Revisited)

The claim " $-b/w$ is Cauchy if w and $b \sim N(0, 1)$ " is true, BUT NotebookLM fails to explain why this matters—the point is that ReLU threshold locations can be extremely large due to heavy tails, which impacts expressivity.

Oversimplified.
The gradient features ∇f_Θ depend on the input x , and NotebookLM implies they are fixed features under small steps, which is true only under the lazy regime.
It fails to mention that this is not true in realistic modern training (feature learning regime).

Good summary of the "optimizer = norm + step size" perspective.
 Correctly describes that GD, SignSGD, Shampoo fall out of constrained norm minimization.

Descent algorithms are revisited from the perspective of optimizing the linear approximation of the loss function, subject to constraints on the step size $\Delta \Theta$.

- Optimization Tension: There is a conflict between wanting large steps for fast convergence and wanting small steps to ensure the local linear approximation remains accurate.
- Optimizer Recipe (Bernstein + Newhouse 2024): A recipe exists where choosing a norm and a step size yields a corresponding optimizer.
- Sign SGD: Derived by minimizing the linear loss approximation under the ℓ_∞ norm constraint ($\|\Delta \Theta\|_\infty \leq \eta$). The resulting step is $\Delta \Theta = -\eta \cdot \text{sgn}(\nabla \mathcal{L}(\Theta))$.
- Gradient Descent (GD revisited): Derived by minimizing the linear loss approximation under the ℓ_2 norm constraint ($\|\Delta \Theta\|_2 \leq \eta$). This minimization yields a step proportional to the negative gradient direction, scaled by η and the unit norm of the gradient. This is mathematically equivalent to minimizing the loss plus a quadratic regularization term $\lambda \|\Delta \Theta\|_2^2$.
- Shampoo Optimizer: Developed by considering optimization in "weight matrix space" and applying a constraint using the spectral norm ($\|\Delta W\|_2 \leq \eta$). This formulation leads to an update $\Delta W = -\eta U V^T$, where $U \Sigma V^T$ is the Singular Value Decomposition (SVD) of the gradient $\nabla \mathcal{L}(W)$. Shampoo is noted for its success in training algorithm competitions.
- Adam/AdamW: Adam is a standard optimizer, and AdamW (with tuned hyperparameters) is considered the default choice, but it is often associated with lazy training behavior.

IV. Norms and Maximal Update Parameterization (μP) Good identification that μP depends on RMS $\rightarrow$ RMS norms and maintains layer-wise scaling consistency.

This topic focuses on developing scaling rules to stabilize training across networks of different widths, achieved by utilizing specialized norms.

- RMS Norm: Defined as $\|x\|_{\text{RMS}} = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}$, representing the root mean square (average size of each entry).
- Induced RMS $\rightarrow$ RMS Norm: This matrix norm, motivated by Xavier initialization, is defined as $\|A\|_{\text{RMS} \rightarrow \text{RMS}} = \sqrt{\frac{d_{\text{out}}}{d_{\text{in}}} \|A\|_2^2}$, where $\|A\|_2$ is the spectral norm, and $d_{\text{in}}/d_{\text{out}}$ are fan-in/fan-out dimensions. This norm is a rescaled version of the spectral norm. Fails to note μP 's practical importance in modern LLM training (LLaMA, PaLM-like scaling).
- Maximal Update Parameterization (μP): μP is a technique focused on choosing the right "units" (or scaling) for hyperparameters so that they can be transferred from smaller networks to larger ones.
 - The core idea is that applying the constraint $\|\Delta W\|_{\text{RMS} \rightarrow \text{RMS}} \leq \eta$ allows for choosing a single hyperparameter η (learning rate) across all layers, but the effective learning rates become layer-specific due to the fan-in and fan-out dimensions.
 - For Adam/Sign SGD, maintaining this constraint implies that the learning rate η should scale as d_{in} with respect to the input dimension, which captures the essence of the μP parameterization.
 - This technique leads to optimum stable training behavior across networks of varying widths, contrasting with standard practice, where the optimum learning rate shifts dramatically with network width changes.

V. Muon Optimizer and Impact Muon indeed comes from RMS $\rightarrow$ RMS norm scaling. It does outperform AdamW on validation loss in some "NanoGPT" experiments.

The Muon optimizer is a practical realization of the scaling concepts derived from the RMS $\rightarrow$ RMS norm.

- Muon Optimizer: Derived by using the induced RMS $\rightarrow$ RMS norm within the optimizer recipe.
- Performance: Muon demonstrates improved performance in "Nano GPT speedruns," achieving lower validation loss against Adam, Distributed Shampoo, and SOAP* over wallclock time. It is considered scalable for large language model (LLM) training.

VI. Feature Learning Conditions

Formal conditions are derived for achieving feature learning, ensuring that layer updates are non-negligible as network width grows.

• Desiderata: Feature learning requires two conditions regarding the scaling of hidden layer vectors h :

1. The RMS norm of the hidden layer output must be constant: $\|h\|_{\text{RMS}} = \Theta(1)$.

2. The RMS norm of the change in the hidden layer output must also be constant: $\|\Delta h\|_{\text{RMS}} = \Theta(1)$.

• Weight Norm Conditions: Meeting these output conditions implies that both the weight matrix norm ($\|W\|_{\text{RMS}} \rightarrow \text{RMS}$) and the change in weight matrix norm ($\|\Delta W\|_{\text{RMS}} \rightarrow \text{RMS}$) must be $\Theta(1)$.

You: I want to learn about these sources using Learning Guide

NotebookLM:

That's a great request! The sources cover advanced concepts in initializing and optimizing neural networks, connecting traditional methods to modern scaling techniques.

Since this is a comprehensive overview, let's start with a plan based on the concepts identified in the roadmap:

1. Linearization and Features: Understanding the "lazy training" assumption using Taylor expansion.
2. Initialization Methods: Exploring how Xavier and He initialization help with network stabilization.
3. Optimization as Constrained Minimization: Deriving optimizers (Sign SGD, Gradient Descent, Shampoo) using norms.
4. Scaling and μP : Discussing the RMS norm and Maximal Update Parameterization (μP) for stable training.

Does that sound like a good starting point, or is there a specific topic you want to dive into first?

You: Let's start with the lazy training assumption. What is it, and why is it important? What do we assume from it?

NotebookLM:

The "lazy training assumption" relates to the feature perspective and Taylor expansion of the network.

What it is: It is the assumption that during training, the parameter weights (Θ) move by very small amounts.

Why it is important & what we assume from it:

OVERSIMPLIFIED: Lazy training emerges in infinitely wide networks with appropriately scaled learning rates, not simply because this is "assumed."

1. Linear Approximation: This assumption allows the network output $f(x, \Theta + \Delta \Theta)$ to be accurately approximated using a Taylor expansion (a "linear approximation at the network").

2. Feature Definition: The linear term in this approximation, $\langle \nabla f, \Delta \Theta \rangle$, shows that the entries of the gradient $\nabla \Theta$ function as features.

3. Kernel/NTK Link: This regime is often seen when training with Adam, where models barely change from their random initialization, linking the process to the Kernel/feature learning perspective and the Neural Tangent Kernel (NTK).

Would you like to explore how this local linear perspective is used to derive different optimizers, like Sign SGD and Gradient Descent?

You: In one or two sentences, how can you help me use these sources?

Small parameter movement is indeed the central idea of lazy training.
Lazy training does justify the first-order Taylor expansion. It's true that in this regime, gradient features become effectively fixed, enabling a kernel interpretation (NTK). These are good high-level statements.

NotebookLM:

I can help you explore the sources by detailing concepts like the lazy training assumption and its relation to the Neural Tangent Kernel (NTK), explaining initialization techniques (Xavier, He) necessary for network stability, and deriving optimization algorithms (Sign SGD, Gradient Descent, Shampoo) from the perspective of constrained minimization using norms. Furthermore, I can clarify the scaling principles of the Maximal Update Parameterization (μP) and the RMS $\rightarrow$ RMS norm which lead to the Muon optimizer and enable stable training across different network widths.

You: Can you go over every initialization, their importance, and key advantages/disadvantages to each?

NotebookLM:

The sources focus on initialization techniques designed to ensure stable signal propagation through the network layers, specifically aiming for standardization of inputs to nonlinearities.

Here is an overview of the initialization concepts discussed, their importance, and characteristics derived from the sources:

I. General Initialization Concepts

Initialization Target

- Correctly states the purpose of initialization: maintain stable signal variance as it propagates.
- Correctly highlights the difference between:
 - Xavier initialization (for tanh/sigmoid-like, fan-in/fan-out symmetry)
 - He initialization (for ReLU, compensating for half the activations being 0)
- Correctly identifies why zero initialization is bad (symmetry $\rightarrow$ no gradient).
- Accurately mentions the Cauchy -b/w corner problem for poorly scaled ReLU initialization.

Details

Importance/Impact

Default/Zero Initialization

Setting all weights to zero.

Disadvantage: If all weights are zero, all inputs from the previous layer are multiplied by zero, resulting in zero gradients, meaning the weights will never move.

Standardization (Input Data)

Initial processing of input data to ensure features are on similar orders of magnitude (e.g., mean 0, variance 1).

Importance: Leads to better conditioning, prevents numerical issues, and ensures large features do not dominate the calculations.

Goal of Weight Initialization

The key observation is that the output activations from one layer become the inputs to the next layer; therefore, the goal is to initialize weights so that the input to the non-linearity is "standardized"—approximately zero-mean and unit-variance. This stability is crucial to preserve the network's expressive power.

II. Specific Weight Initialization Schemes

The sources detail two primary methods for weight initialization (W_i), aiming for unit-variance input to the non-linearity based on the input dimension (d , or fan-in).

Initialization

Target Scaling

Key Observation/Advantage

I think Xavier Init. comes in 2 forms with the $\sqrt{6/\text{fan_avg}}$ coefficient

Xavier Initialization

Weights W are sampled from a normal distribution: $W \sim N(0, 1/d)$, where d is the fan-in.

Advantage: Designed to ensure the variance of the sum of weighted inputs remains consistent across layers (approaching "unit-variance"). This technique motivated the development of the RMS → RMS norm later used in sophisticated optimizers.

He Initialization

Specifically designed for the ReLU non-linearity. Weights W are sampled from a normal distribution: $W \sim N(0, 2/d)$.

Advantage: Corrects the variance reduction caused by ReLU, as the ReLU output is zero approximately half the time.

III. Bias Initialization

NotebookLM describes the formulas but not the reasoning:

Initialization Method

Xavier → Variance preservation through both forward and backward passes. This prevents signal explosion and prevents gradients from vanishing.

Details

He → Variance preservation only for forward pass using ReLU. The ReLU kills half the signal → variance doubles → need factor 2.

Initialize at 0

NotebookLM only says "unit variance" without showing why these formulas work.

Setting the bias parameter to zero.

Small Positive Number

Bias initialization section is extremely shallow

Using a small number such as 0.01.

NotebookLM lists options ("0", "0.01", "treat as Xavier") but never discusses why biases are often initialized to zero (it doesn't cause symmetry problems), why positive bias is sometimes used for ReLU (prevent initial dying ReLUs), or why modern LLMs often set many biases to zero by design (e.g., no bias in LayerNorm, no bias in Attention projections).

Treat as Xavier

Applying the Xavier framework to the bias initialization.

IV. Disadvantage of Poor Initialization (ReLU Case)

If the initialization of weights (w) and biases (b) is poor (e.g., $w \sim N(0, 1)$ and $b \sim N(0, 1)$), the point where the ReLU nonlinearity "bends" (the "corner" at $-b/w$) follows a Cauchy distribution.

- The Cauchy distribution is "0-mean" but has infinite variance.
- If the initialized data range (e.g., 100 to 200) does not overlap well with the non-linearity's corner, the network loses its expressive power. For instance, if data is in the range 100 to 200, the probability of a corner landing there might be extremely low (e.g., 0.16%), meaning only a tiny fraction of units might utilize the nonlinearity, causing loss of expressive power.

You: Can you go through each of the optimization algorithms in depth, and talk about their code ideas and the problems each intend to solve?

NotebookLM:

The sources derive optimization algorithms, not through empirical testing, but mathematically by applying constraints (norms) to the goal of minimizing the loss using a local linear approximation.

The fundamental tension that all descent algorithms must solve is the balance between taking large steps to converge fast and taking small steps to ensure the linear approximation of the loss still holds.

The general recipe (from Bernstein & Nemirovski, 1994) for choosing a norm and choosing a step size (η) and this

- Correctly identifies the general recipe:
Minimize the linearized loss $\langle \nabla L(\theta), \Delta \theta \rangle$ subject to a norm constraint $\|\Delta \theta\| \leq \eta$.
- Correctly states that different choices of norms → different optimizers.
- Provides correct high-level interpretations of:
 - SignSGD → ℓ_∞ constraint
 - GD → ℓ_2 constraint
 - Shampoo → matrix/spectral-norm-aware constraint
 - Muon → RMS→RMS-norm constraint motivated by μP / Maximal Update Param.

The general recipe (from Bernstein + Newhouse 2024) is: Choose a norm and choose a step size (η), and this combination yields a specific optimizer by minimizing the linear loss approximation $\langle \nabla \mathcal{L}(\Theta), \Delta \Theta \rangle$ subject to the constraint $\|\Delta \Theta\| \leq \eta$.

The sources detail four optimizers derived from this framework: Sign SGD, Gradient Descent (GD), Shampoo, and Muon.

I. Sign Stochastic Gradient Descent (Sign SGD)

Core Idea/Derivation: Sign SGD is derived by minimizing the linearized loss approximation subject to an ℓ_∞ norm constraint on the parameter update $\Delta \Theta$:

$$\|\Delta \Theta\|_\infty \leq \eta \argmin \langle \nabla \mathcal{L}(\Theta), \Delta \Theta \rangle$$

Code Idea (Update Rule): The ℓ_∞ constraint means that the maximum absolute value of any entry in $\Delta \Theta$ must be less than or equal to η . To minimize the inner product (i.e., maximize the negative inner product), the optimal step $\Delta \Theta^*$ must move as far as possible ($\pm \eta$) in the direction opposite to the gradient.

- If the gradient entry $\nabla \mathcal{L}(\Theta)[j]$ is positive, $\Delta \Theta[j]$ is set to $-\eta$.
- If the gradient entry $\nabla \mathcal{L}(\Theta)[j]$ is negative, $\Delta \Theta[j]$ is set to $+\eta$.

The resulting update rule is: $\Delta \Theta^* = -\eta \cdot \text{sgn}(\nabla \mathcal{L}(\Theta))$.

Problem Solved/Context: The sources revisit Adam/Sign SGD. Later analysis shows how the scaling of η in Adam/Sign SGD needs to be adjusted based on the layer input dimension (d_{in}) to maintain stable update behavior across different network widths, linking it to the essence of μP .

Missing the geometric intuition

- ℓ_1 norm chooses the coordinate with the worst-case contribution $\rightarrow$ leads to sign updates.
- ℓ_2 norm spreads the update proportionally across all coordinates $\rightarrow$ classical GD.
- Spectral norm constrains the most amplified direction of a layer $\rightarrow$ Shampoo approximates full-matrix preconditioning.
- RMS $\rightarrow$ RMS norm normalizes updates by fan-in/fan-out geometry $\rightarrow$ stable scaling across widths.

In my opinion the geometric intuition is essential for understanding why the method works.

II. Gradient Descent (GD)

Core Idea/Derivation: Gradient Descent (GD) is derived by minimizing the linearized loss approximation subject to an ℓ_2 norm constraint on the parameter update $\Delta \Theta$:

$$\|\Delta \Theta\|_2 \leq \eta \argmin \langle \nabla \mathcal{L}(\Theta), \Delta \Theta \rangle$$

Code Idea (Update Rule): Using the Cauchy-Schwartz inequality, the inner product is minimized when the vectors are anti-aligned. The optimal step $\Delta \Theta^*$ must point in the direction of the negative gradient and be scaled to meet the ℓ_2 constraint η .

$\Delta \Theta^* = -\eta \frac{\nabla \mathcal{L}(\Theta)}{\|\nabla \mathcal{L}(\Theta)\|_2}$. The sources note this constrained formulation is mathematically equivalent to the unconstrained minimization of the loss plus a quadratic regularization term $\lambda \|\Delta \Theta\|_2^2$.

Problem Solved/Context: GD is the classical descent algorithm. When revisited through the linear perspective, it provides a strong foundation for understanding how the constraint choice dictates the direction and scale of the update.

III. Shampoo Optimizer

Core Idea/Derivation: Shampoo moves beyond treating parameters (Θ) as a flat vector and instead considers them organized as Weight matrices (W). It uses the same minimization recipe but applies a different matrix norm related to the geometry of the weight matrix.

$\|\Delta W\|_2 \leq \eta \argmin \langle \nabla \mathcal{L}(W), \Delta W \rangle$ Here, $\|\Delta W\|_2$ refers to the spectral norm (the largest singular value of the matrix ΔW). The inner product $\langle A, B \rangle$ for matrices is defined as the trace of $A^T B$ (the Frobenius inner product).

Code Idea (Update Rule): If the gradient of the loss, $\nabla W_L(W)$, is decomposed using Singular Value Decomposition (SVD) as $U \Sigma V^T$:

$$\Delta W \leftarrow -\eta U V^T.$$

Problem Solved/Context: Shampoo attempts to use knowledge of the weight matrix structure in the update constraint, rather than just the vector norm. A variant of the Shampoo optimizer (Dahl 2023) won a training algorithm competition.

IV. Muon Optimizer

Core Idea/Derivation: Muon is derived by substituting the ℓ_2 or spectral norm constraint with a highly scaled norm motivated by initialization principles: the Induced RMS $\rightarrow$ RMS norm. This norm, $\|A\|_{\text{RMS} \rightarrow \text{RMS}}$, is a rescaled version of the spectral norm $\|A\|_2$, where d_1 is the fan-in and d_2 is the fan-out:

$$\| \Delta W \|_{\text{RMS} \rightarrow \text{RMS}} = \frac{d_2}{d_1} \| \Delta W \|_2.$$

When this norm is used in the minimization recipe: $\| \Delta W \|_{\text{RMS} \rightarrow \text{RMS}} \leq \gamma \argmin \langle \nabla W_L(W), \Delta W \rangle$

Problem Solved/Context: The core problem Muon and its underlying principle, Maximal Update Parameterization (μP), address is the instability of hyperparameter choice when scaling network width.

- **Scaling Stability:** By choosing the RMS $\rightarrow$ RMS norm, one can use a single hyperparameter η across all layers, but the effective learning rate automatically becomes layer-specific due to the fan-in (d_1) and fan-out (d_2) dimensions.
- **Performance:** Muon is noted for being scalable for large language model (LLM) training. It has demonstrated improved validation loss compared to Adam and Distributed Shampoo in "Nano GPT speedruns" when measured against wallclock time.

You: Explain how normalization leads to stable training, and specifically what specific normalization heuristics are covered in the sources.

NotebookLM:

Normalization leads to stable training by ensuring that the signals and updates propagating through the network layers maintain desirable statistical properties, which in turn preserves the network's expressive power and improves optimization efficiency.

I. How Normalization Leads to Stable Training

The stability achieved through normalization stems from two primary areas:

1. Handling Input Data and Preventing Dominance:

- Normalization, specifically standardization of data, is crucial because it ensures features are on similar orders of magnitude.
- If input data is not standardized, large features might dominate the calculations.
- Proper standardization leads to better conditioning and avoids numerical issues, which contributes significantly to stability.

2. Maintaining Expressive Power During Signal Propagation (Initialization):

- The key observation in network initialization is that output activations from one layer become the input to the next layer.

- The goal of initialization is to ensure that the input to each non-linearity starts out "standardized," meaning it is approximately zero-mean and unit-variance.
- If poor initialization is used, such as initializing weights and biases from $N(0,1)$, the critical point (or "corner") of the non-linearity (like ReLU, at $-b/w$) follows a Cauchy distribution, which has infinite variance.
- If the data range does not overlap well with the non-linearity's corner, the units might not utilize the non-linearity, causing the network to lose expressive power. Initialization heuristics, therefore, are normalization strategies applied to weights to prevent variance explosion/collapse across layers, thus keeping the network expressive.

II. Specific Normalization Heuristics Covered in the Sources

The sources cover both data normalization heuristics and parameter initialization heuristics (which stabilize internal layer norms):

A. Initialization Heuristics (Parameter Normalization)

These methods prescribe how to scale weights based on layer dimensions (fan-in, d) to stabilize the variance of the input signal to the non-linearity:

Heuristic

Goal / Non-Linearity

Normalization Scaling

Xavier Initialization

Designed to ensure the variance of the sum of weighted inputs is consistent across layers, making the input to the non-linearity approximately zero-mean and unit-variance.

Weights W are sampled from $N(0, 1/d)$, where d is the fan-in.

He Initialization

Specifically designed for the ReLU non-linearity.

Since ReLU outputs are zero approximately half the time, the variance must be boosted. Weights W are sampled from $N(0, 2/d)$.

B. Normalization for Scaling and Optimization (μP)

For large networks and advanced optimization, specialized norms are used to define stable scaling properties:

Heuristic

Definition

Role in Stability

RMS Norm

Defined for a vector x as

Induced RMS $\rightarrow$ RMS Norm

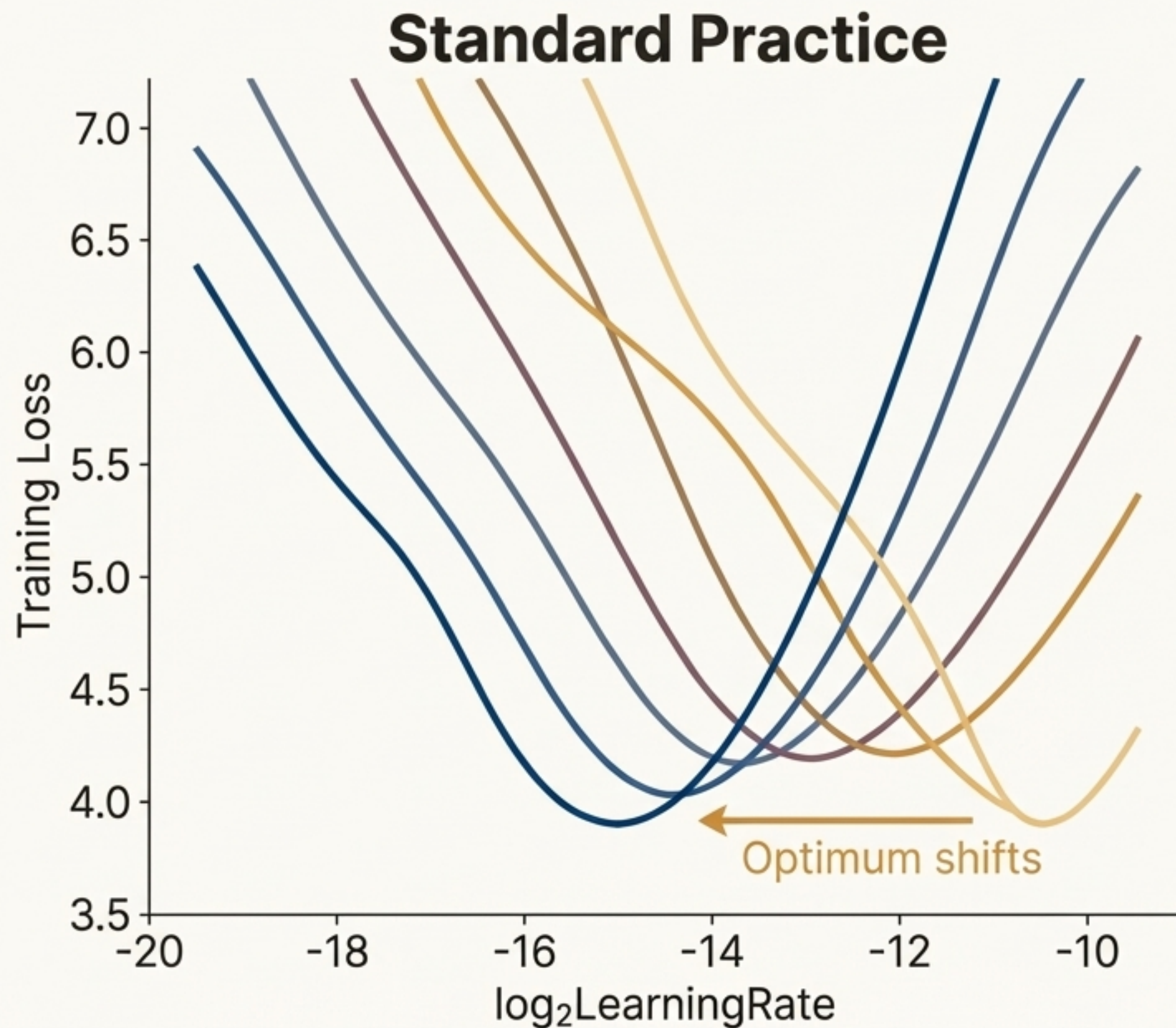
This matrix norm is motivated by Xavier initialization. It is a rescaled version of the spectral norm $\|A\|_2$: $\text{RMS} \rightarrow \text{RMS} = d_{\text{out}} / d_{\text{in}} \|A\|_2$.

This normalization is key to the Maximal Update Parameterization (μP). By constraining weight updates (ΔW) using this norm, the effective learning rate is automatically layer-specific, enabling the choice of a single hyperparameter η .

and norm, the effective learning rate is automatically layer-specific, enabling the choice of a single hyperparameter η across all layers that results in optimum stable training behavior across networks of varying widths, unlike standard practice. This concept is used to derive the Muon optimizer.

Below, NotebookLM created a slideshow explaining all the concepts relating to optimizers, with created graphics and visuals to help students actually understand concepts with diagrams. It generated pretty quickly (~5 minutes), and created a pretty densely informed slideshow that I think would be worth it for a student trying to synthesize concepts quickly and efficiently.





The Hyperparameter Scaling Problem

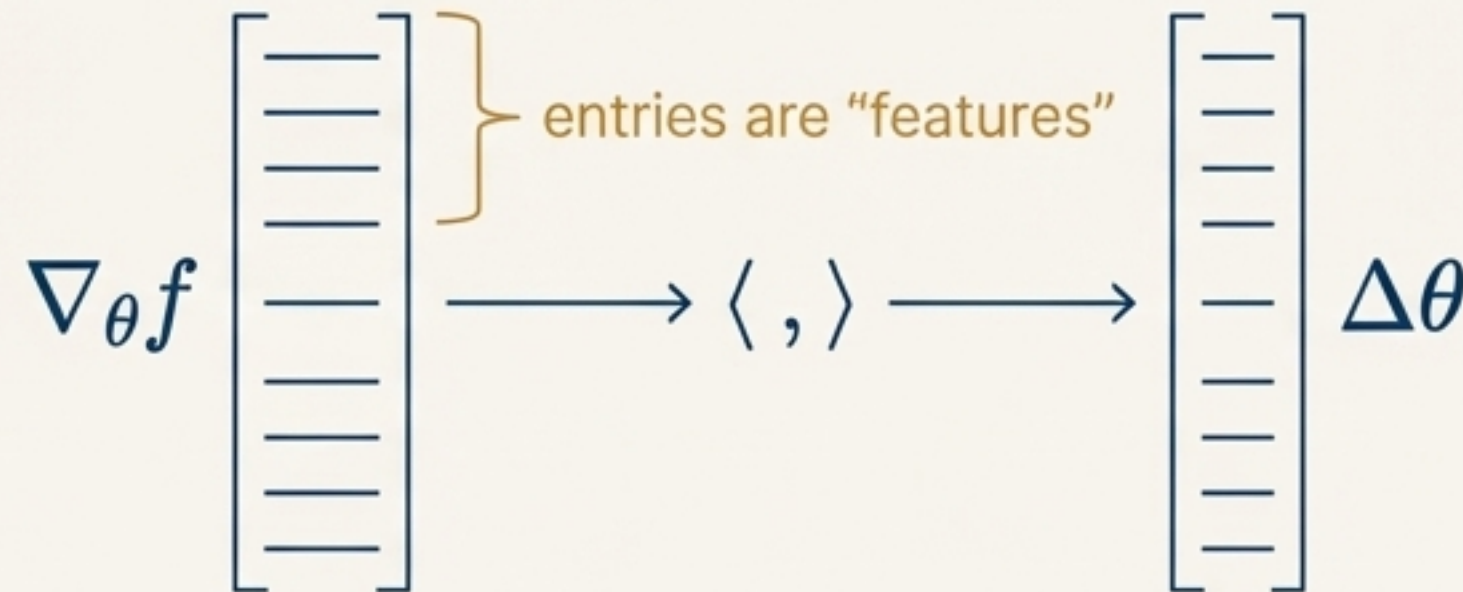
Training large neural networks is a significant investment of time and resources. A key challenge is that hyperparameters, like the learning rate, are extremely sensitive to model architecture. What works for a small model often fails dramatically for a larger one, forcing expensive, repeated tuning.

Our goal: Find a principled way to choose hyperparameters that are stable across model scales.

A New Perspective: The Local Linearity of Neural Networks

To understand how to control a network's training dynamics, we can approximate its behavior locally. Using a first-order Taylor expansion, we can view the network $f(x, \theta)$ as a linear function of the parameter change $\Delta\theta$ for any small step. This approximation, $f(x, \theta + \Delta\theta) \approx f(x, \theta) + \langle \nabla_{\theta} f, \Delta\theta \rangle$, is our key analytical tool. It holds true under the “lazy training” assumption, where weights move by small amounts, which is common in practice. The **gradient** $\nabla_{\theta} f$ acts as a vector of “features” that determine how a change in parameters affects the output.

$$f(x, \theta + \Delta\theta) \approx f(x, \theta) + \langle \nabla_{\theta} f, \Delta\theta \rangle$$



Optimizers Emerge from a Single, Simple Principle

With our linear perspective, optimization becomes a clear task. At each step, we want to find a parameter update $\Delta\theta$ that maximally decreases the loss. However, we must constrain the step size $\|\Delta\theta\| \leq \eta$ to ensure our linear approximation remains valid. This creates a universal "recipe" for an optimizer:

The Core Tension

- Want large steps to converge fast.
- Want small steps so the approximation holds.

$$\operatorname{argmin}_{\|\Delta\theta\| \leq \eta} \langle \nabla_{\theta} L, \Delta\theta \rangle$$

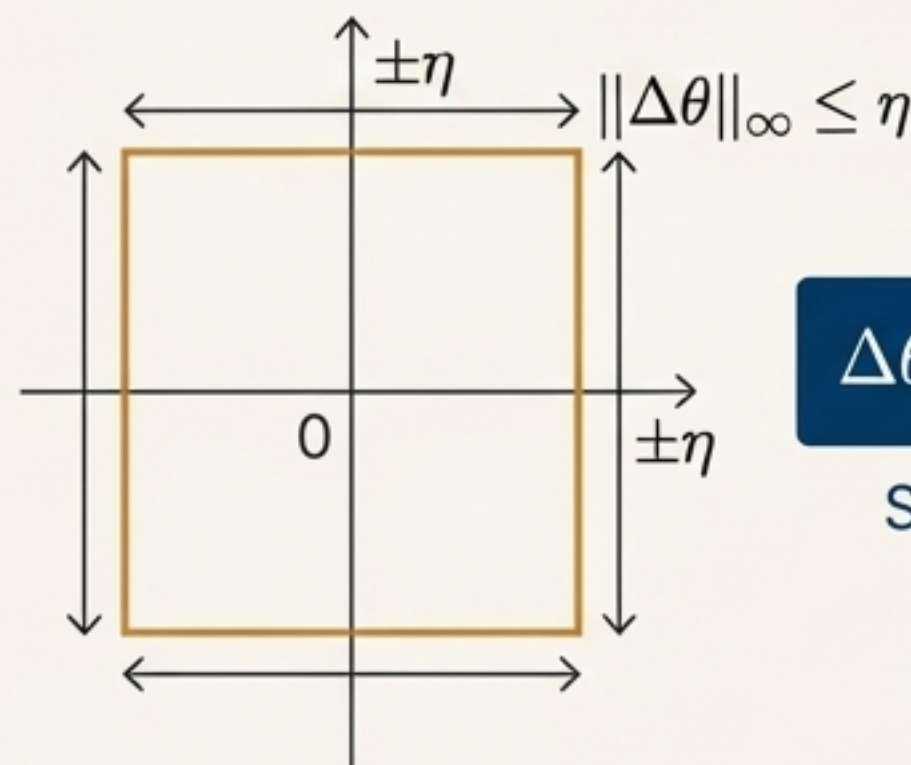
This is a universal recipe:
Choose a norm, get an optimizer.

The Choice of Norm Dictates the Update Rule

Let's see our recipe in action. The geometry of the constraint $\|\Delta\theta\| \leq \eta$ determines the optimal direction for $\Delta\theta$.

Choice 1: L-infinity Norm

This constrains each parameter update $\Delta\theta[j]$ independently. The solution yields the SignSGD update rule:

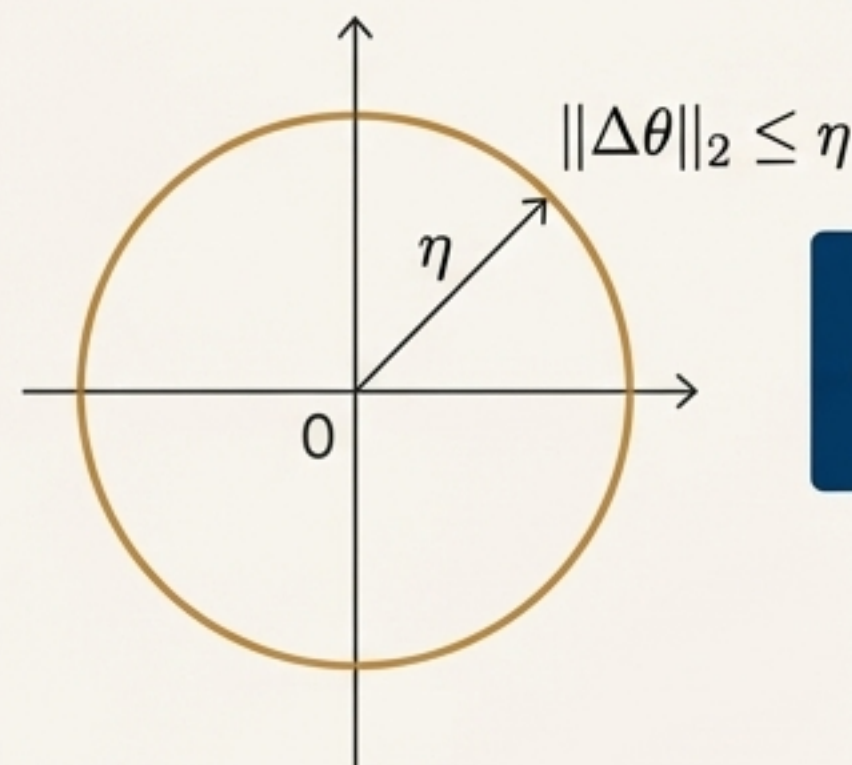


$$\Delta\theta = -\eta \cdot \text{sign}(\nabla_\theta L)$$

SignSGD update rule.

Choice 2: L2 Norm

This constrains the total Euclidean length of the update vector. The solution yields the standard Gradient Descent update rule:



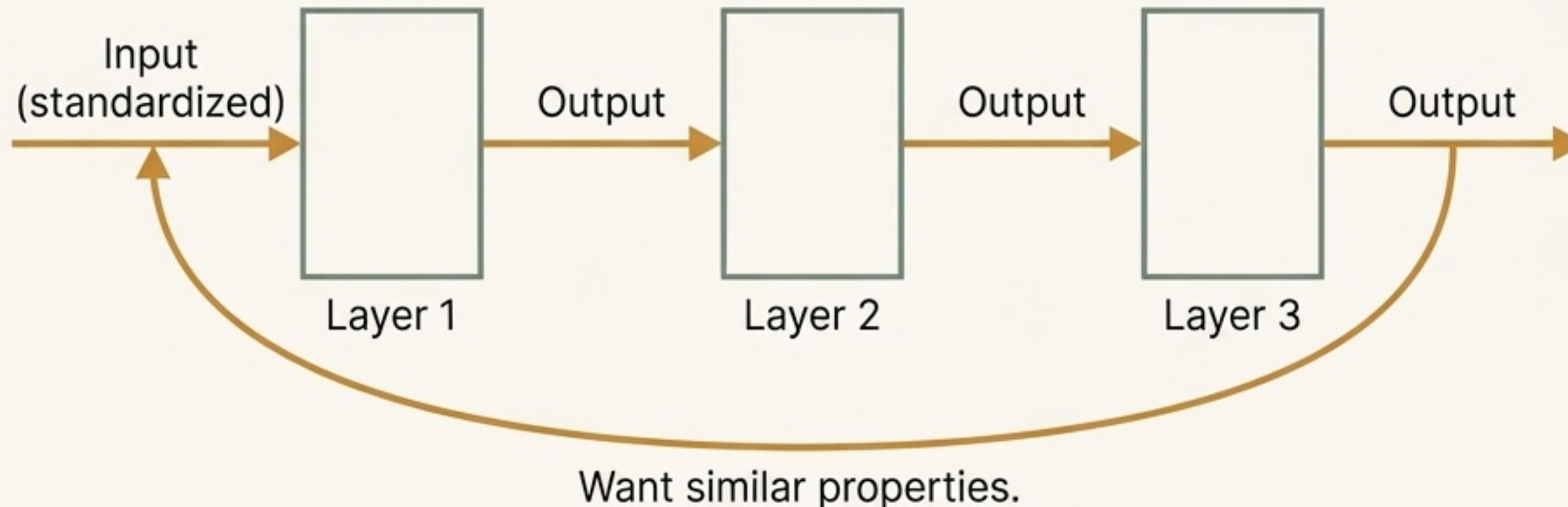
$$\Delta\theta = -\eta \cdot \frac{\nabla_\theta L}{\|\nabla_\theta L\|_2}$$

Gradient Descent
update rule.

Initialization's True Goal: Preserving Signal Integrity

Why do we carefully initialize weights? The fundamental reason is that the output activations of one layer are the inputs to the next. For a deep network to learn effectively, the statistical properties of the signal—specifically its scale—must be preserved as it propagates forward.

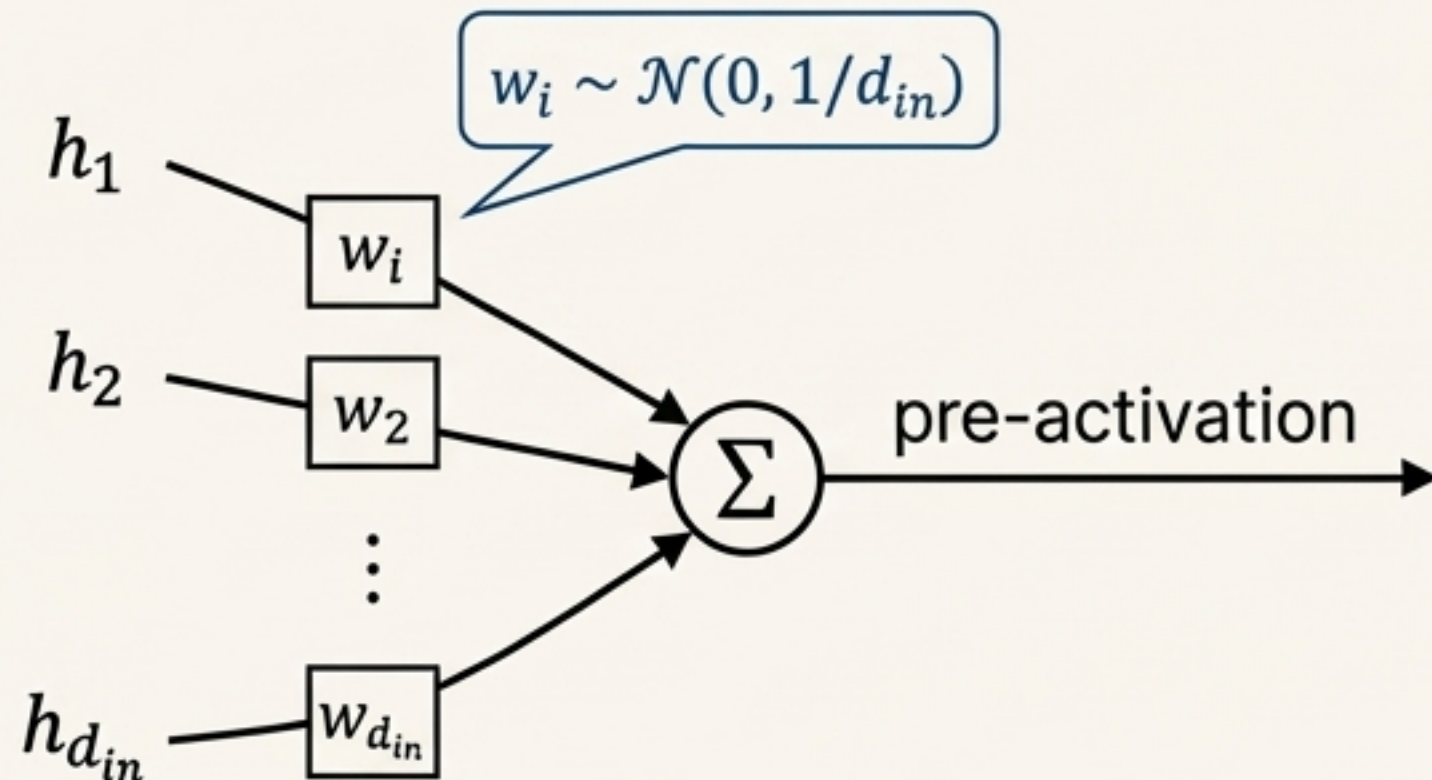
If the signal vanishes or explodes, the network loses its expressive power. The goal is to initialize weights such that for a standardized input (mean 0, variance 1), the output is also approximately standardized.



Xavier Initialization is a Statement About the RMS Norm

Standard initialization schemes like Xavier formalize the goal of preserving signal scale. For a layer with d_{in} inputs, Xavier initialization scales the weights w_i by $\frac{1}{\sqrt{d_{in}}}$. This ensures that if the inputs $h_i - h_i$ are standardized, the pre-activation $\sum w_i h_i$ has a variance of approximately 1.

How can we express this goal—“the average size of each entry is 1”—using the language of norms? This is precisely what the **Root Mean Square (RMS) norm** measures:



$$\|\mathbf{x}\|_{\text{RMS}} = \sqrt{\frac{1}{d} \sum x_i^2} = \frac{1}{\sqrt{d}} \|\mathbf{x}\|_2$$

Xavier's goal is $\|\mathbf{h}_{\text{out}}\|_{\text{RMS}} \approx 1$.

Unifying the Mysteries: What if the Optimizer Used the Initialization Norm?

We have uncovered two deep principles, expressed in a common language:

1. **Optimizers:** Emerge from a recipe $\operatorname{argmin}_{\|\Delta\theta\| \leq \eta} \langle \nabla_{\theta} L, \Delta\theta \rangle$. The choice of norm is critical.
2. **Initialization:** Aims to preserve signal scale, which is perfectly captured by requiring $\|h\|_{\text{RMS}} \approx 1$.



This suggests a profound connection. What happens if we apply the optimizer recipe not to parameter vectors, but to weight *matrices*, and use a norm inspired by the RMS principle from initialization?

The Revelation: The Induced RMS Norm Creates Principled Scaling

We need a matrix norm that measures how a weight matrix $\mathbf{W}$ transforms a vector in RMS space. This is the induced RMS→RMS norm. It can be shown that this norm is simply a rescaling of the standard spectral norm:

$$\|\mathbf{W}\|_{RMS \rightarrow RMS} = \sqrt{\frac{d_{in}}{d_{out}}} * \|\mathbf{W}\|_2$$

Now, let's use this norm in our optimizer recipe:

$$\|\Delta \mathbf{W}\|_{RMS \rightarrow RMS} \leq \eta$$

This single constraint on the update size, applied uniformly across all layers, has a remarkable effect. It implies a layer-specific constraint on the spectral norm of the update:

This is the core of Maximal Update Parameterization (μP). A single hyperparameter η produces adaptive, layer-specific effective learning rates.

1. Define the Norm

$$\|\mathbf{W}\|_{RMS \rightarrow RMS} = \sqrt{\frac{d_{in}}{d_{out}}} * \|\mathbf{W}\|_2$$

2. Apply to Optimizer Recipe

$$\|\Delta \mathbf{W}\|_{RMS \rightarrow RMS} \leq \eta$$

3. The Implication (μP)

$$\|\Delta \mathbf{W}\|_2 \leq \eta * \sqrt{\frac{d_{out}}{d_{in}}}$$

μP Provides a Scaling Rule for Any Optimizer

μP is not an optimizer itself, but a “parameterization”—a rule for how to set hyperparameters as the model architecture changes. We can use it to adapt standard optimizers like Adam or SignSGD.

Let’s derive the rule for SignSGD. The update is $\Delta W = -\eta \cdot S$, where $S = \text{sign}(\nabla_W L)$. We want to maintain $\|\Delta W\|_{\text{RMS} \rightarrow \text{RMS}} \leq \gamma$.

For a single data point (batch size 1), S is a rank-1 matrix with entries ± 1 . Its spectral norm is $\|S\|_2 = \sqrt{d_{\text{in}} \cdot d_{\text{out}}}$. Plugging this into our μP constraint gives:

$\eta \cdot \sqrt{d_{\text{in}}/d_{\text{out}}} \cdot \sqrt{d_{\text{in}} \cdot d_{\text{out}}} \leq \gamma \Rightarrow \eta \cdot d_{\text{in}} \leq \gamma$. This tells us the learning rate η for each layer should be scaled as:

Learning Rate Scaling

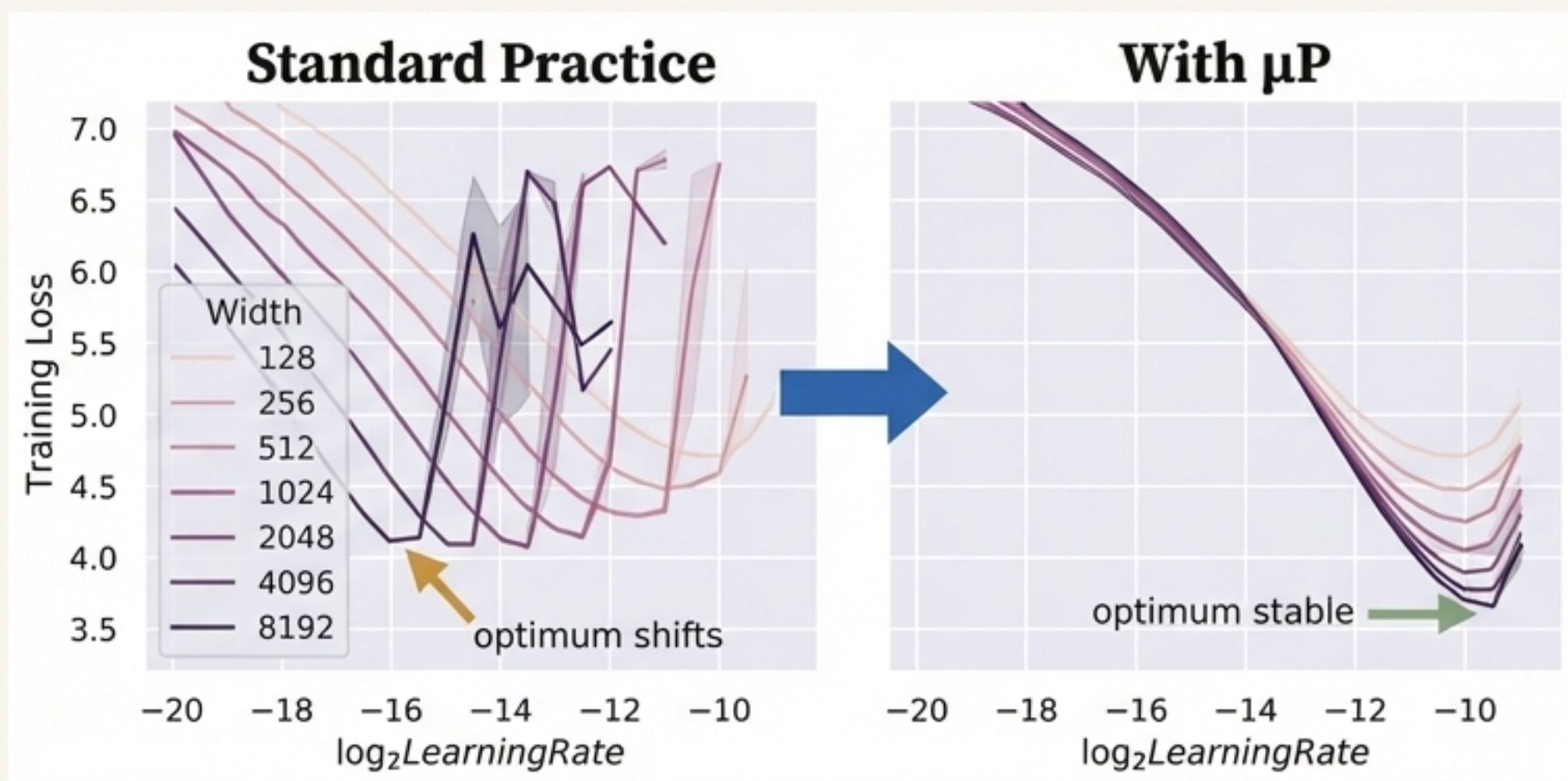
$$\eta \propto \frac{1}{d_{\text{in}}}$$

Before μP	With μP
Layer 1 LR: η	Layer 1 LR: $\eta \propto \frac{1}{d_{\text{in}_1}}$
Layer 2 LR: η	Layer 2 LR: $\eta \propto \frac{1}{d_{\text{in}_2}}$
Layer 3 LR: η	Layer 3 LR: $\eta \propto \frac{1}{d_{\text{in}_3}}$

The Proof: Stable Optimal Learning Rates Across All Scales

Does this principle work in practice? Yes. By applying μP , the optimal learning rate becomes remarkably stable as network width changes. We can now find the best learning rate on a small, cheap-to-train model and confidently transfer it to a massive one.

This was a known issue with large models like GPT-3, where learning rates had to be manually manually decayed for wider architectures. μP provides the theoretical foundation for this scaling.

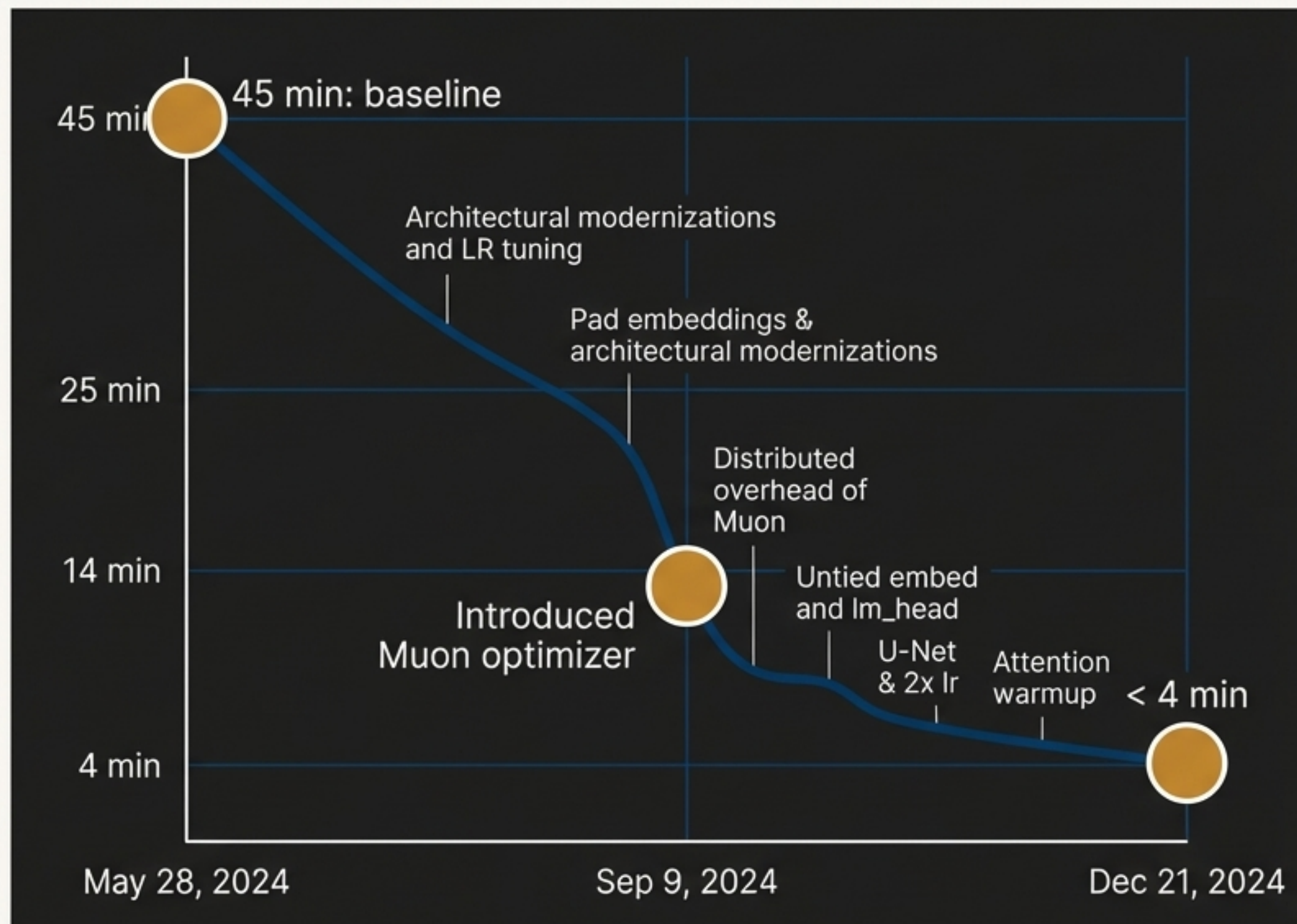


Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 or "GPT-3"	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

The Impact: A 10x Reduction in Training Time

These principles are not merely theoretical. They are the foundation for new, highly efficient optimizers like Muon.

In a 'speedrun' to train a NanoGPT model, a series of improvements were made over several months. The single largest leap in performance came from introducing the Muon optimizer, which is built on the μ P principles of principled scaling. The total training time was reduced from 45 minutes to under 4 minutes.



The Conditions for Stable Feature Learning

At a deeper level, μP works by ensuring the network is in a regime of stable feature learning as its width grows. This requires satisfying two conditions for the hidden states $\mathbf{h}_l$ at each layer l :

1. Stable Activation Scale: The magnitude of the activations must remain constant.
2. Stable Update Scale: The magnitude of the change in activations per step must also remain constant.

Desideratum 1: Stable Activations

$$\|\mathbf{h}_l\|_{\text{RMS}} = \Theta(1)$$

Desideratum 2: Stable Updates

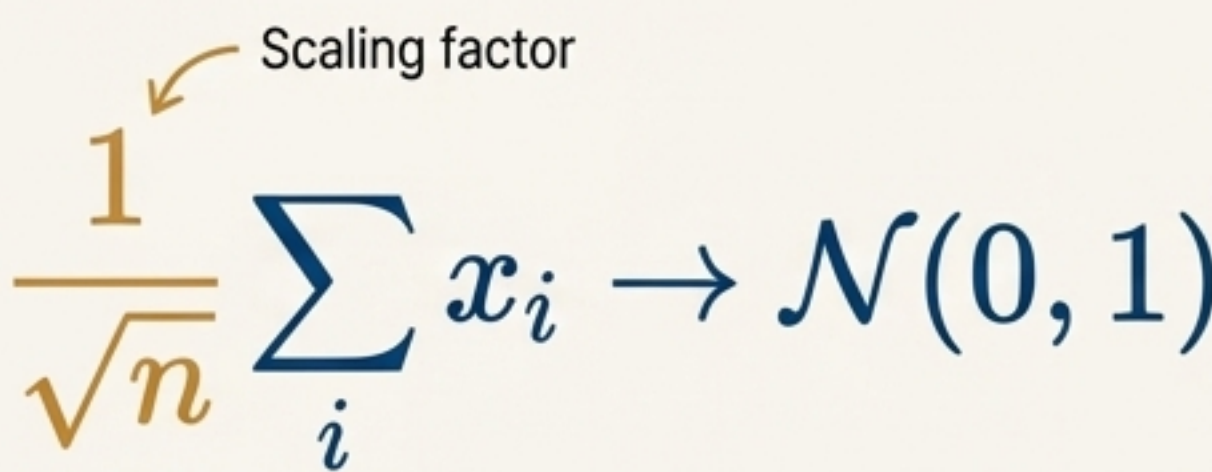
$$\|\Delta\mathbf{h}_l\|_{\text{RMS}} = \Theta(1)$$

It can be shown that these two conditions are met if and only if the weight matrices $\mathbf{W}$ and their updates $\Delta\mathbf{W}$ also have a constant scale in the RMS $\rightarrow$ RMS norm. This is precisely what our optimizer recipe enforces. These are achieved if $\|\mathbf{W}_l\|_{\text{RMS}\rightarrow\text{RMS}} = \Theta(1)$ and $\|\Delta\mathbf{W}_l\|_{\text{RMS}\rightarrow\text{RMS}} = \Theta(1)$.

The Final Insight: Parameterization is About Choosing the Right Units

The core lesson is that proper parameterization is about finding the right “units” or “scaling factors” so that the underlying system behaves consistently as its scale changes. In statistics, $1/\sqrt{n}$ is the correct factor to make the sum of random variables converge to a non-trivial limit.

In deep learning, **Maximal Update Parameterization (μP)** provides the correct scaling for hyperparameters. It defines a “stable” parameterization where insights from small models can be directly transferred to large ones, finally taming the complexity of training at scale.

Statistics: Central Limit Theorem	Deep Learning: μP
$\frac{1}{\sqrt{n}} \sum_i x_i \rightarrow \mathcal{N}(0, 1)$	 Small Model Optimal LR = η^* Scaling to Larger Models Large Model Optimal LR = η^* Stable & Transferable
The $1/\sqrt{n}$ factor ensures the sum converges to a standard normal distribution.	With μP, the optimal hyperparameter (e.g., Learning Rate η^*) discovered on a small model is directly transferable to a large model.